Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Master Thesis – Structure Proposal A+B

# Template A+B: Design, Implementation, Evaluation with Research Questions

by

Author's Name

7th October, 2026

University of Kaiserslautern-Landau
Department of Computer Science
Distributed Systems Lab

Examiner: Prof. Dr.-Ing. Jens B. Schmitt
M. Sc. Supervisor Name

# About This Proposal

**Organising idea.** Template A's chapters unchanged: the thesis tells the story of a system, from problem through design and implementation on four edges to how well it works. From Template B it takes the research questions as the backbone of the evaluation. They are stated in the introduction, every metric in the methodology is tied to one, the Results chapter has one section per question ending with an explicit answer, and the conclusion answers each in turn. Modelled on general thesis guidance [1, 2], systems measurement papers such as MeshInsight [3], and the RQ convention of empirical studies [4].

**What changed from A.** Section 1.3 states the questions in full; Section 6.1 maps metrics to them; Chapter 7 is regrouped into RQ1–RQ4; Discussion opens with a synthesis across the questions; Conclusion answers each one.

| Ch. | Title | approx. pages |
|---|---|---|
| 1 | Introduction | 5 |
| 2 | Background | 10 |
| 3 | Related Work | 6 |
| 4 | Design of WADM | 12 |
| 5 | Implementation on Four Edges | 14 |
| 6 | Evaluation Methodology | 10 |
| 7 | Results (RQ1–RQ4) | 20 |
| 8 | Discussion | 6 |
| 9 | Conclusion and Future Work | 3 |
| A–D | Appendices | as needed |

Page counts are a rough budget, not a department rule. The unibz guidelines advise keeping a chapter under about 20 pages and splitting it above 30 [2].

**How to read the rest.** The table of contents is the proposed structure. Under each heading is a short note on what to write there, and a *Material* line pointing to the place in the repository that already holds it (`results/` and `plots/` are short for `benchmarks/results/` and `benchmarks/results/plots/`).

## Front matter (unchanged from the department template)

- **Title page** and **Eigenständigkeitserklärung**, as generated by `rptuma.cls`. The declaration requires disclosing generative-AI use beyond grammar and style.

- **Abstract** (English): the problem (deception at the web edge has an unknown cost), the approach (one mechanism on four edges, levelled to identical behaviour), the method (paired benchmark on three measurement planes) and the headline numbers.
- **Zusammenfassung**: the same abstract in German.
- **Contents**, **List of Figures**, **List of Tables**.
- **Glossary** (optional, the class's `glossar` environment): WADM, honeytoken, edge, tier, VU, iteration, SLO, cgroup, LuaJIT, PUC Lua, proxy-wasm.

In the department template's own example, websites are cited through an archived copy (`web.archive.org`); do the same for any web source in the list at the end.

# Contents


**1 Introduction** **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Questions . . . 1
- 1.4 Contributions . . . 1
- 1.5 Thesis Outline . . . 2

**2 Background** **3**
- 2.1 Cyber Deception, Honeypots and Honeytokens . . . 3
- 2.2 Web Application Reconnaissance . . . 3
- 2.3 Reverse Proxies and Their Request Phases . . . 3
- 2.4 Extension Runtimes . . . 3
- 2.5 Performance Measurement Fundamentals . . . 3

**3 Related Work** **4**
- 3.1 Honeytokens and Decoys . . . 4
- 3.2 Application-Layer Deception without Code Changes . . . 4
- 3.3 Proxy and Service-Mesh Overhead . . . 4
- 3.4 Benchmarking Methodology in Systems Security . . . 4
- 3.5 Positioning . . . 4

**4 Design of WADM** **5**
- 4.1 Requirements and Threat Model . . . 5
- 4.2 Architecture . . . 5
- 4.3 Configuration Model . . . 5
- 4.4 Honeytoken Kinds . . . 5
  - 4.4.1 HTML Comments . . . 5
  - 4.4.2 HTTP Headers . . . 5
  - 4.4.3 Cookies . . . 6
  - 4.4.4 Decoy Paths . . . 6
  - 4.4.5 Form Fields . . . 6
- 4.5 The Fake SQL-Injection Trap . . . 6
- 4.6 Detection and Response Pipeline . . . 6
- 4.7 The Canonical Mechanism . . . 6

**5 Implementation on Four Edges** **7**
- 5.1 Common Structure . . . 7

5.2 OpenResty (Reference Edge) . . . 7
5.3 Envoy with a Lua Filter . . . 7
5.4 Envoy with a WebAssembly Filter . . . 7
5.5 Apache httpd with mod_lua . . . 8
5.6 Levelling the Edges . . . 8
5.7 Verifying Equivalence . . . 8

**6 Evaluation Methodology** **9**
6.1 Goals and Metrics . . . 9
6.2 Testbed . . . 9
6.3 Workload . . . 9
6.4 Measurement Planes . . . 9
6.5 Paired Three-Tier Design . . . 10
6.6 Run Validity . . . 10
6.7 Statistics and Presentation . . . 10

**7 Results** **11**
7.1 RQ1: Latency Added . . . 11
7.1.1 Median Latency per GET across Rates . . . 11
7.1.2 Tail Latency . . . 11
7.1.3 Request Position and Reconnects . . . 11
7.1.4 Answer to RQ1 . . . 11
7.2 RQ2: CPU and Capacity . . . 11
7.2.1 CPU per Request . . . 11
7.2.2 Cost Attribution . . . 12
7.2.3 The Cost of Measuring . . . 12
7.2.4 Per-Mechanism Cost and Page Size . . . 12
7.2.5 Capacity and the Binding Resource . . . 12
7.2.6 Answer to RQ2 . . . 12
7.3 RQ3: Functional Correctness . . . 12
7.3.1 Response Integrity under Load . . . 12
7.3.2 Detection Recall and False Positives . . . 13
7.3.3 Response-Shape Matrix . . . 13
7.3.4 Answer to RQ3 . . . 13
7.4 RQ4: Influence of the Extension Runtime . . . 13
7.4.1 Per-Kind Operation Cost . . . 13
7.4.2 The SQLi Trap: Scan Depth versus Decoding . . . 13
7.4.3 Case Study: Stability of Apache mod_lua . . . 13
7.4.4 Answer to RQ4 . . . 13

**8 Discussion** **14**
8.1 Synthesis across Research Questions . . . 14
8.2 Choosing an Edge for Deception . . . 14
8.3 Where the Cost Comes From . . . 14

8.4 Deployment Implications . . . . . 14
8.5 Threats to Validity . . . . . 14
8.6 Limitations . . . . . 14

**9 Conclusion and Future Work** **15**
9.1 Summary . . . . . 15
9.2 Future Work . . . . . 15

**A Configuration Reference** **16**

**B Parity-Check Request Set** **17**

**C Reproducing the Benchmarks** **18**

**D Additional Result Tables** **19**

# 1 Introduction

Why deception at the web edge is worth studying, and what this thesis contributes.

## 1.1 Motivation

Attackers inspect a web application's HTML, cookies and hidden paths before attacking it. Honeytokens planted there raise high-signal alerts, because no legitimate user touches them. An edge proxy can plant them without changing the application.

## 1.2 Problem Statement

Nobody has measured what this costs. Which edge technology should host the deception, and what does it cost in latency, CPU, capacity and correctness?

## 1.3 Research Questions

**RQ1** How much latency does WADM add per request on each edge, and how does that change with load? **RQ2** What does it cost in CPU and capacity, and where does that cost arise? **RQ3** Does it break what the server did without it, and does it detect what it should? **RQ4** How much of the answer depends on the extension runtime (LuaJIT, PUC Lua, WebAssembly)?

## 1.4 Contributions

(i) WADM: one configuration, five honeytoken kinds and a fake SQL-injection trap, implemented on four edges; (ii) behavioural and tuning levelling, with an automated parity check; (iii) a paired, three-plane benchmark harness; (iv) measured costs and where they come from.

## 1.5 Thesis Outline

One line per chapter, noting that Chapter 7 answers RQ1–RQ4 in order.

# 2 Background

The concepts a reader needs before the design makes sense.

## 2.1 Cyber Deception, Honeypots and Honeytokens

Spitzner's definition [5, 6]; honeytokens as data with no legitimate use; why false positives are rare.

## 2.2 Web Application Reconnaissance

What attackers read and tamper with: HTML source, response headers, cookies, `robots.txt`, hidden form fields, SQL-injection probes.

## 2.3 Reverse Proxies and Their Request Phases

Request hooks, header and body filters, body buffering, upstream connection pools.

## 2.4 Extension Runtimes

LuaJIT (OpenResty, Envoy Lua), PUC Lua 5.1 (Apache `mod_lua`), WebAssembly through proxy-wasm on V8 (Rust). Interpreted, JIT-compiled and compiled code compared.

**Material:** `CONTEXT.md` (tech stack table)

## 2.5 Performance Measurement Fundamentals

Latency percentiles, open- and closed-loop load, coordinated omission, cgroup CPU accounting [7].

# 3 Related Work

Prior work, grouped by theme, ending with where this thesis sits.

## 3.1 Honeytokens and Decoys

Decoy documents [8], honeywords [9], and deceptive web artefacts such as HTML comments, `robots.txt` entries and invisible links [10].

## 3.2 Application-Layer Deception without Code Changes

Kahlhofer and Rass review 19 techniques for deploying deception without developer involvement, reverse proxies among them [11].

## 3.3 Proxy and Service-Mesh Overhead

Sidecar overhead studies [3]; Lua versus WebAssembly filters in Envoy.

## 3.4 Benchmarking Methodology in Systems Security

Common benchmarking flaws [12, 13].

## 3.5 Positioning

No prior work measures deception cost across edge runtimes while holding behaviour identical.

# 4 Design of WADM

The mechanism on a conceptual level, independent of any one edge.

## 4.1 Requirements and Threat Model

The operator controls the edge but not the application. The attacker is a reconnaissance or scanning client. Non-goals: blocking traffic, real authentication.

## 4.2 Architecture

Client → edge → static origin. One shared `config.json`, an in-memory attacker-IP store, and the alert log as WADM's only output.

**Material:** `CONTEXT.md` (architecture diagram)

## 4.3 Configuration Model

Per-token `enabled` switch, exact or `/*` paths, and plans precompiled per path when the configuration loads.

**Material:** `config.json`, `CONTEXT.md` (field tables)

## 4.4 Honeytoken Kinds

### 4.4.1 HTML Comments

Injected before `</body>`; detected when the trigger keyword appears in the query.

### 4.4.2 HTTP Headers

A decoy response header; detected when its value is replayed in a later request.

### 4.4.3 Cookies

`Set-Cookie` bait; detected on tamper, since a browser replays the value unchanged.

### 4.4.4 Decoy Paths

Advertised as a hidden link or in `robots.txt`; any request to the path matches.

### 4.4.5 Form Fields

A hidden `<input>` before `</form>`; detected on tamper.

## 4.5 The Fake SQL-Injection Trap

The edge answers `POST /api/login` itself: a fake MySQL error that reflects the payload on a signature hit, a canned denial otherwise. Signatures are literal strings, not regular expressions.

## 4.6 Detection and Response Pipeline

Parse once, scan, strip the parameter so the origin never sees it, record the IP, then write one alert with CR/LF neutralised against log forgery.

## 4.7 The Canonical Mechanism

Rules every edge must follow: client IP from the socket peer, raw path, ordered query segments, and one plain-find splice primitive.

**Material:** `docs/EDGE_LEVELING.md` (“Behavioural equivalence and latency pass”)

# 5 Implementation on Four Edges

How the design maps onto each proxy, and what made the four comparable.

## 5.1 Common Structure

A table mapping each edge's request hook, header filter and body filter, so the subsections below need only describe the differences.

## 5.2 OpenResty (Reference Edge)

Configuration compiled once in `init_by_lua`, detection in `access_by_lua`, injection in the header and body filters, attacker IPs in a `lua_shared_dict`.

**Material:** `nginx/README.md`, `nginx/nginx.conf`

## 5.3 Envoy with a Lua Filter

`injection.lua`, handing the request path to the response phase via dynamic metadata, body buffering.

**Material:** `envoy_scripts/README.md`

## 5.4 Envoy with a WebAssembly Filter

Rust proxy-wasm: configuration compiled in `on_configure`, fewer host calls, a byte-level splice. A proxy-wasm filter cannot answer locally from the body callback.

**Material:** `wasm-filter/README.md`, `wasm-filter/src/lib.rs`

## 5.5 Apache httpd with mod_lua

The shared `wadm.lua` core and its three hook scripts; `LuaScope` and `LuaCodeCache`.

**Material:** `apache_scripts/README.md`

## 5.6 Levelling the Edges

Product defaults levelled before any comparison: upstream keep-alive, access logs, code caching. Baselines are the WADM configuration minus WADM. The residual differences are listed in a table.

**Material:** `docs/EDGE_LEVELING.md`, `docs/THESIS_NOTES.md` (2026-09-17, 2026-09-18)

## 5.7 Verifying Equivalence

`parity_check.py` sends 13 requests to every edge and diffs the responses, the alert lines and the requests that reached the origin.

**Material:** `benchmarks/parity_check.py`

# 6 Evaluation Methodology

How the costs were measured, and why the measurements can be trusted.

## 6.1 Goals and Metrics

A table mapping each metric to its question. RQ1: latency added (median and p95). RQ2: CPU per request, capacity. RQ3: share of intact responses, alert recall, false positives. RQ4: in-edge timer cost per kind.

## 6.2 Testbed

The VM, the cpuset layout, guest tuning, image versions.

**Material:** `docs/BENCHMARK_METHODOLOGY.md` §4

## 6.3 Workload

k6 iterations of nine requests: a priming `GET`, four `GET` probes and four SQLi `POST`s. Williams-balanced order [14]; arrival rates of 1, 10 and 100 iterations/s.

**Material:** `benchmarks/test.js`

## 6.4 Measurement Planes

In-edge timers (µs), end-to-end latency from k6 (ms), CPU per request from cgroup `cpu.stat`.

## 6.5 Paired Three-Tier Design

Bare, silent and instrumented tiers run minutes apart in balanced order, over 5 replicates.

**Material:** `docs/BENCHMARK_METHODOLOGY.md` §7.1

## 6.6 Run Validity

The `assess_run()` gates, the origin-CPU negative control, and detection of host freezes.

**Material:** `benchmarks/wadm_timings.py`, `docs/THESIS_NOTES.md` (2026-10-07)

## 6.7 Statistics and Presentation

Medians of paired differences, the range over replicates, sign agreement, and a 5%-trimmed mean for the timers.

# 7 Results

One section per research question, each ending with its answer.

## 7.1 RQ1: Latency Added

### 7.1.1 Median Latency per GET across Rates

+109 to +218 µs per `GET` (13–23%), resolved in every `GET` cell.

**Material:** `results/paired_headline.md`, `plots/paired_latency_added.png`

### 7.1.2 Tail Latency

p95 grows more than the median on every edge. Apache at 100 iterations/s collapses.

### 7.1.3 Request Position and Reconnects

The first request after the think-time sleep pays a wake-up cost. Apache closes the connection after the trap's 500.

**Material:** `docs/THESIS_NOTES.md` (2026-09-21, §5b), `plots/paired_latency_shift.png`

### 7.1.4 Answer to RQ1

One paragraph: the number per edge, how it changes with load, and how certain it is.

## 7.2 RQ2: CPU and Capacity

### 7.2.1 CPU per Request

CPU added per request: OpenResty +33, Envoy+Lua +41, WASM +123, Apache +200 µs.

### 7.2.2 Cost Attribution

The in-edge timers cover 5–22% of that CPU; the rest is VM entry and exit, body filters and log writes.

**Material:** `plots/cost_attribution.png`

### 7.2.3 The Cost of Measuring

The timing log lines alone cost about as much as WADM itself on OpenResty and Envoy+Lua.

### 7.2.4 Per-Mechanism Cost and Page Size

The `GET` path against the trap; injection cost grows linearly with body size up to 768 KiB.

**Material:** `results/mechanisms.md`, `plots/mechanism_cpu.png`, `plots/size_sweep.png`

### 7.2.5 Capacity and the Binding Resource

Share of bare capacity kept with silent WADM: OpenResty 71%, WASM 71%, Envoy+Lua 84%, Apache 30%, each reported with the resource that limited it.

**Material:** `results/capacity_summary.md`, `plots/capacity.png`

### 7.2.6 Answer to RQ2

CPU and capacity per edge, and where the cost arises.

## 7.3 RQ3: Functional Correctness

### 7.3.1 Response Integrity under Load

Share of intact responses per probe and rate. Apache is the only edge that falls below 100%.

**Material:** `results/paired_functional.md`

### 7.3.2 Detection Recall and False Positives

`WADM ALERT` and `WADM TRAP` lines counted against expected; zero on bare.

### 7.3.3 Response-Shape Matrix

Gzip-encoded bodies and range requests are corrupted on all four edges. HEAD, bodies over 1 MiB and ETags each fail on some of them.

**Material:** `results/functional_matrix.md`, `benchmarks/functional_matrix.py`

### 7.3.4 Answer to RQ3

What breaks, on which edge, and whether detection is complete.

## 7.4 RQ4: Influence of the Extension Runtime

### 7.4.1 Per-Kind Operation Cost

In-edge timer distributions per honeytoken kind and edge.

**Material:** `plots/paired_internal_timers.png`

### 7.4.2 The SQLi Trap: Scan Depth versus Decoding

In the encoding-by-outcome 2×2, decoding costs 0.5–2.5 µs on the Lua edges and nothing on WASM; scan depth is visible only on Apache's interpreted PUC Lua.

**Material:** `results/sqli.md`, `docs/THESIS_NOTES.md` (2026-09-21)

### 7.4.3 Case Study: Stability of Apache mod_lua

Segfaults driven by cumulative requests in the body filter, and the cost of fixing them with `LuaScope conn`.

**Material:** `docs/BENCHMARK_METHODOLOGY.md` §4b–4c, `plots/apache_remediation.png`

### 7.4.4 Answer to RQ4

How much of RQ1–RQ3 is explained by the runtime rather than by the deception logic.

# 8 Discussion

What the results mean, and how far they can be trusted.

## 8.1 Synthesis across Research Questions

One table of edges against the four answers.

## 8.2 Choosing an Edge for Deception

A trade-off summary per edge: latency, CPU, capacity, robustness.

## 8.3 Where the Cost Comes From

Runtime entry and exit, body buffering and log writes, not the detection logic.

## 8.4 Deployment Implications

Compressed bodies, range requests, preserved ETags, body-size limits.

## 8.5 Threats to Validity

A single VM host, synthetic traffic, moving image tags, timer resolution.

## 8.6 Limitations

POST-body inspection exists on only two edges; no real attacker traffic; no TLS or HTTP/2.

# 9 Conclusion and Future Work

## 9.1 Summary

RQ1–RQ4 answered in one or two sentences each, then the overall contribution.

## 9.2 Future Work

POST-body inspection on all four edges, injecting into compressed bodies, a field study with real attackers, TLS and HTTP/2.

# A Configuration Reference

Every `config.json` field per honeytoken kind, and the SQLi trap's fields.

**Material:** `CONTEXT.md`

# B Parity-Check Request Set

The 13 requests and the documented exclusions.

# C Reproducing the Benchmarks

Commands, machine configuration, run durations.

**Material:** `docs/BENCHMARK_METHODOLOGY.md` §3–4

# D Additional Result Tables

Per-rate tables and figures.

**Material:** `benchmarks/results/plots/by_rate/`

# Sources Used for This Proposal


[1] S. Intille, "Organizing a thesis." https://ccis.northeastern.edu/home/intille/teaching/advising/organizationTips.htm. Accessed: 07.10.2026.

[2] Free University of Bozen-Bolzano, Faculty of Computer Science, "Master in computer science – guidelines for the thesis." https://guide.unibz.it/assets/graduation/Computer-Science/Master/Guidelines-Thesis-Master-2022.pdf. Approved 03.03.2010. Accessed: 07.10.2026.

[3] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, "Dissecting service mesh overheads." arXiv:2207.00592, 2022.

[4] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[5] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.

[6] L. Spitzner, "Honeytokens: The other honeypot." SecurityFocus, 2003.

[7] R. Jain, *The Art of Computer Systems Performance Analysis*. Wiley, 1991.

[8] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, "Baiting inside attackers using decoy documents," in *Security and Privacy in Communication Networks (SecureComm)*, pp. 51–70, 2009.

[9] A. Juels and R. L. Rivest, "Honeywords: Making password-cracking detectable," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pp. 145–160, 2013.

[10] N. Virvilis, B. Vanautgaerden, and O. S. Serrano, "Changing the game: The art of deceiving sophisticated attackers," in *6th International Conference on Cyber Conflict (CyCon)*, pp. 87–97, 2014.

[11] M. Kahlhofer and S. Rass, "Application layer cyber deception without developer interaction," in *3rd Workshop on Active Defense and Deception (ADnD), co-located with IEEE EuroS&P*, 2024. arXiv:2405.12852.

[12] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.

[13] G. Heiser, "Systems benchmarking crimes." `https://gernot-heiser.org/benchmarking-crimes.html`. Accessed: 07.10.2026.

[14] E. J. Williams, "Experimental designs balanced for the estimation of residual effects of treatments," *Australian Journal of Scientific Research, Series A*, vol. 2, no. 2, pp. 149–168, 1949.